

Baconian HOL: An Isabelle/HOL Formalization Scaffold for Higher-Order Metaphysics

July 15, 2026

1 Purpose of the scaffold

The current Isabelle/HOL development is a first, buildable scaffold for formalizing higher-order metaphysics in the setting of Bacon’s *A Philosophical Introduction to Higher-Order Logics*. The governing choice is that Isabelle/HOL is used as the metalanguage. The Bacon-style higher-order language is represented inside Isabelle as an object language.

This choice is deliberate. Isabelle/HOL already has simple types, lambda abstraction, higher-order quantification, and equality. If we used all of those directly as the language being studied, many of the distinctions that matter in Bacon’s setting would disappear. In particular, object-language identity, object-language implication, object-language abstraction, and object-language quantification would be identified with Isabelle’s own metalogical equality, implication, lambda abstraction, and quantification. That would be too extensional for many of the intended metaphysical applications.

The present development therefore deep-embeds a small typed object language. The development currently contains:

- object-language types for individuals, propositions, and arrows;
- object-language variables, constants, application, and lambda abstraction;
- object-language equality, negation, conjunction, disjunction, and implication;
- object-language universal and existential quantification;
- a typing judgment for object-language terms;
- renaming and substitution operations over de Bruijn variables;

- preservation lemmas for renaming, substitution, and one-step beta contraction;
- Hilbert-style theoremhood for the minimal higher-order logic H ;
- a Classicism extension C containing the Boolean identities and the Classicist identities;
- a first modal-logicism derivation layer, including restricted necessitation and a zero-ary equivalence-closure extension;
- a vector/zeta equivalence layer for fresh de Bruijn variables and higher-type Equivalence;
- a standalone typed beta-eta conversion relation, together with bridge lemmas from conversion to biconditional theoremhood in H , C , CE , and CEV ;
- an S4-style modal derivation layer, including equality symmetry, equality transitivity, and the modal K , T , and 4 schemata;
- an abstract semantic layer, including applicative structures, semantic environments, recursive interpretation of object terms, typed evaluation, semantic substitution and conversion preservation, full H -soundness for `valid_in_context`, and relative soundness theorems for C , CE , and CEV ;
- a completeness interface, including semantic consequence for assumption lists, local soundness for `H_derivable` and `C_derivable`, and countermodel-property bridge theorems for completeness;
- a canonical-theory layer, including H/C Lindenbaum-Henkin extensions, canonical truth lemmas, substitutional term-model semantics, H/C Henkin-validity and term-model completeness biconditionals, explicit H/C countermodel packages, and strengthened CE/CEV local consequence machinery for the remaining completeness work;
- a Caie names layer, including the type correspondence for individuals, propositions, first-order properties, and property classifiers; an uninterpreted counterfactual connective; the maximal-consistency, name, and haecceity vocabulary; deep-embedded Appendix C theorem statements; applied typing lemmas; verified definitional conversion bridges for `heq`, down-projection, up-projection, and down-similarity; and a local Caie derivability wrapper over explicit finite assumption lists.
- a finite Caie M^* model layer, checking the current two-world model core of Caie's Proposition 29.

The session builds with Isabelle2025-2:

```
isabelle build -D .
```

2 Bacon-style notation

2.1 Types

Bacon's typed languages use simple types. I will write t for the type of propositions and ι for a background type of individuals. If σ and τ are types, then

$$\sigma \rightarrow \tau$$

is the type of functions or operations from expressions of type σ to expressions of type τ . Thus a predicate of individuals may be written as having type $\iota \rightarrow t$, a binary relation on individuals as $\iota \rightarrow \iota \rightarrow t$, and an operation from propositions to propositions as $t \rightarrow t$.

The scaffold also defines an order function on types:

$$\text{ord}(\iota) = 0, \quad \text{ord}(t) = 0, \quad \text{ord}(\sigma \rightarrow \tau) = \max(\text{ord}(\sigma) + 1, \text{ord}(\tau)).$$

This is not yet used in a major theorem, but it is a useful bookkeeping device for later separating first-order, second-order, and genuinely higher-order fragments.

2.2 Terms

In standard Bacon-style notation, terms are typed expressions. Typical typing claims have the form

$$\Gamma \vdash M : \sigma,$$

where Γ is a context assigning types to variables, M is a term, and σ is a type.

The core term-forming operations represented so far are these:

x	variable	$\Gamma \vdash x : \sigma,$
c_σ	constant of type σ	$\Gamma \vdash c_\sigma : \sigma,$
MN	application	$\frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash MN : \tau},$
$\lambda x^\sigma.M$	abstraction	$\frac{\Gamma, x : \sigma \vdash M : \tau}{\Gamma \vdash \lambda x^\sigma.M : \sigma \rightarrow \tau},$
$M =_\sigma N$	typed equality	$\frac{\Gamma \vdash M : \sigma \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M =_\sigma N : t},$
$\sim A$	negation	$\frac{\Gamma \vdash A : t}{\Gamma \vdash \sim A : t},$
$A \& B$	conjunction	$\frac{\Gamma \vdash A : t \quad \Gamma \vdash B : t}{\Gamma \vdash A \& B : t},$
$A \rightarrow B$	implication	$\frac{\Gamma \vdash A : t \quad \Gamma \vdash B : t}{\Gamma \vdash A \rightarrow B : t},$
$\forall x^\sigma A$	higher-order universal quantification	$\frac{\Gamma, x : \sigma \vdash A : t}{\Gamma \vdash \forall x^\sigma A : t}.$

Here the quantifier is higher-order in the broad sense: σ may itself be a functional type, such as $t \rightarrow t$, $t \rightarrow t$, or $(t \rightarrow t) \rightarrow t$.

2.3 Beta contraction

The central syntactic operation currently formalized is beta contraction:

$$(\lambda x^\sigma.M)N \rightarrow_\beta M[N/x].$$

The corresponding preservation theorem says:

$$\text{if } \Gamma \vdash (\lambda x^\sigma.M)N : \tau \text{ and } (\lambda x^\sigma.M)N \rightarrow_\beta M[N/x], \text{ then } \Gamma \vdash M[N/x] : \tau.$$

This is the first syntactic safety property one wants before adding proof systems or model theory.

3 Isabelle-side representation

3.1 Session layout

The Isabelle session is the repository root:

Baconian_HOL/

It currently contains:

ROOT	session declaration,
README.md	short project description and build command,
STATUS.md	verified, conditional, and open-result ledger,
CITATION.cff	software citation metadata,
LICENSE	BSD 2-Clause license,
check_isabelle.sh	portable one-command session build,
Bacon_Types.thy	object-language types,
Bacon_Syntax.thy	object-language terms,
Bacon_Typing.thy	typing judgment and type inference,
Bacon_Substitution.thy	renaming and substitution,
Bacon_Beta.thy	beta contraction and type preservation,
Bacon_Deduction.thy	the minimal higher-order logic H ,
Bacon_Abbreviations.thy	Bacon-Dorr identity schemas,
Bacon_Classicism.thy	the Classicism extension C ,
Bacon_Modal.thy	modal-logicism definitions and sanity checks,
Bacon_Modal_Derivations.thy	restricted necessitation and equivalence closure,
Bacon_Zeta.thy	vector/zeta equivalence infrastructure,
Bacon_Conversion.thy	standalone beta-eta equivalence and proof-system bridge,
Bacon_S4.thy	local CEV derivability, equality transfer, and modal S4,
Bacon_Semantics.thy	abstract applicative semantics and soundness,
Bacon_Completeness.thy	semantic consequence and countermodel bridges to completeness,
Bacon_Canonical.thy	canonical theories and Henkin-completeness infrastructure,
Bacon_Caie.thy	Caie names vocabulary, Appendix C statements, and conversion bridges,
Bacon_Caie_Mstar.thy	finite M^* table model for Caie's Proposition 29 core.

3.2 Object-language types in Isabelle

The object-language type datatype is:

```
datatype otype =
  Ind
| Prop
| Arr otype otype
```

The Isabelle theory installs an infix notation for `Arr sigma tau`. Mathematically, this is the object-language arrow

$$\sigma \Rightarrow_o \tau.$$

In Isabelle source, that same symbol is written with Isabelle’s symbolic escape syntax as `\<rightarrow>\<^sub>o`: `\<rightarrow>` renders as $\rightarrow$, and `\<^sub>o` renders the following `o` as a subscript. The subscript marks this as the object-language arrow, not Isabelle/HOL’s own function arrow.

Thus the correspondence with the Bacon notation is:

Bacon-style notation	Isabelle constructor	Report notation
ι	<code>Ind</code>	<code>Ind</code>
t	<code>Prop</code>	<code>Prop</code>
$\sigma \rightarrow \tau$	<code>Arr sigma tau</code>	$\sigma \Rightarrow_o \tau$

3.3 Object-language terms in Isabelle

The object-language term datatype is:

```
datatype oterm =
  Var nat
| Const string otype
| App oterm oterm
| Lam otype oterm
| Eq otype oterm oterm
| Neg oterm
| Conj oterm oterm
| Disj oterm oterm
| Imp oterm oterm
| Forall otype oterm
| Exists otype oterm
```

The key design point is that variables are represented by de Bruijn indices. There are no named bound variables in the Isabelle representation. Instead, `Var 0` means the most recently bound variable, `Var 1` means the next outer bound variable, and so on. This makes alpha-equivalent formulas definitionally identical in Isabelle.

For example, the Bacon expressions

$$\lambda x^\sigma.x \quad \text{and} \quad \lambda y^\sigma.y$$

are represented by the same Isabelle term:

`Lam  $\sigma$  (Var 0).`

In Isabelle syntax this is:

`Lam sigma (Var 0)`

3.4 Contexts

The Isabelle context type is:

`type_synonym ctx = "otype list"`

The intended reading is that the head of the list gives the type of `Var 0`, the next entry gives the type of `Var 1`, and so on. Thus the Bacon context

$$x_0 : \sigma_0, x_1 : \sigma_1, \dots, x_{n-1} : \sigma_{n-1}$$

is represented in reverse binding order as a list

$$[\sigma_{n-1}, \dots, \sigma_1, \sigma_0],$$

if x_{n-1} is the nearest enclosing binder.

The lookup function is:

```
lookup Gamma n =
  (if n < length Gamma then Some (Gamma ! n) else None)
```

So `lookup( $\Gamma, n$ ) =  $\sigma$` says that de Bruijn variable n has type σ in context Γ .

3.5 Typing judgment

The main typing judgment is the inductive predicate:

```
inductive has_type :: "ctx => oterm => otype => bool"
  ("_ \<turnstile> _ : _" [50, 50, 50] 50)
```

In prose I write this as

$$\Gamma \vdash M : \sigma.$$

It is an object-language typing judgment defined inside Isabelle/HOL. It is not Isabelle's own type checker.

The rules correspond directly to the Bacon-style rules above:

Bacon-style clause	Isabelle rule
$\Gamma \vdash x : \sigma$	<code>has_type.Var</code>
$\Gamma \vdash c_\sigma : \sigma$	<code>has_type.Const</code>
$\Gamma \vdash MN : \tau$	<code>has_type.App</code>
$\Gamma \vdash \lambda x^\sigma.M : \sigma \rightarrow \tau$	<code>has_type.Lam</code>
$\Gamma \vdash M =_\sigma N : t$	<code>has_type.Eq</code>
$\Gamma \vdash \sim A : t$	<code>has_type.Neg</code>
$\Gamma \vdash A \& B : t$	<code>has_type.Conj</code>
$\Gamma \vdash A \rightarrow B : t$	<code>has_type.Imp</code>
$\Gamma \vdash \forall x^\sigma A : t$	<code>has_type.Forall</code>

4 Translation table

The following table gives the working correspondence between ordinary Bacon-style syntax and the Isabelle encoding.

Bacon-style notation	Isabelle representation	Comment
ι	<code>Ind</code>	individual base type
t	<code>Prop</code>	proposition base type
$\sigma \rightarrow \tau$	<code>Arr sigma tau</code>	object-language arrow type
x	<code>Var n</code>	de Bruijn variable
c_σ	<code>Const c sigma</code>	typed object constant
MN	<code>App M N</code>	object-language application
$\lambda x^\sigma.M$	<code>Lam sigma M'</code>	M' uses <code>Var 0</code> for x
$M =_\sigma N$	<code>Eq sigma M N</code>	typed object equality
$\sim A$	<code>Neg A</code>	object negation
$A \& B$	<code>Conj A B</code>	object conjunction
$A \rightarrow B$	<code>Imp A B</code>	object implication

$A \leftrightarrow B$	<code>ObjIff A B</code>	abbreviation for both implications
$\forall x^\sigma A$	<code>Forall sigma A'</code>	A' uses <code>Var 0</code> for x
$\top$	<code>ObjTrue</code>	defined as $\forall p^t(p \rightarrow p)$
$\perp$	<code>ObjFalse</code>	defined as <code>Neg ObjTrue</code>
$\Box_o A$	<code>ObjBox A</code>	defined as $A =_t \top_o$
$\Diamond_o A$	<code>ObjDiamond A</code>	defined as $\sim \Box_o \sim A$
$A \leq_o B$	<code>ObjEntails A B</code>	defined as $\Box_o(A \rightarrow B)$
$M[N/x]$	<code>subst0 N M'</code>	capture-avoiding substitution for <code>Var 0</code>
$(\lambda x^\sigma.M)N \rightarrow_\beta M[N/x]$	<code>App (Lam sigma M) N</code> <code>$\rightarrow_\beta$ subst0 N M</code>	one-step beta contraction

5 Current metatheorems

5.1 Type inference soundness and completeness

Besides the inductive typing judgment, the scaffold defines a deterministic type inference function:

$$\text{infer}(\Gamma, M) \in \{\text{none}\} \cup \{\text{some } \sigma : \sigma \in \text{Ty}\}.$$

In Isabelle this is:

```
fun infer_type :: "ctx => oterm => otype option"
```

Two basic metatheorems have been proved:

$$\text{infer}(\Gamma, M) = \sigma \quad \Rightarrow \quad \Gamma \vdash M : \sigma$$

and

$$\Gamma \vdash M : \sigma \quad \Rightarrow \quad \text{infer}(\Gamma, M) = \sigma.$$

Together these show that the executable type checker and the inductive typing judgment agree.

5.2 Uniqueness of types

The development proves:

$$\Gamma \vdash M : \sigma \ \& \ \Gamma \vdash M : \tau \quad \Rightarrow \quad \sigma = \tau.$$

This is useful immediately in beta preservation, since inversion on an application gives both the function type and the argument type, and uniqueness lets Isabelle identify the domain type of the lambda expression with the type of the argument.

5.3 Renaming preservation

The development defines capture-respecting renaming:

$$\text{rename}(\rho, M).$$

The binder case lifts the renaming function so that the newly bound variable is left fixed:

$$\rho^\dagger(0) = 0, \quad \rho^\dagger(n+1) = \rho(n) + 1.$$

The theorem proved is:

$$\begin{aligned} & \Gamma \vdash M : \tau, \\ & \forall n, \alpha \ (\text{lookup}(\Gamma, n) = \alpha \Rightarrow \text{lookup}(\Delta, \rho(n)) = \alpha) \\ & \Rightarrow \quad \Delta \vdash \text{rename}(\rho, M) : \tau. \end{aligned}$$

Weakening at the front of a context is an immediate corollary, implemented as

$$\text{shift}(M) = \text{rename}(\lambda n. n + 1, M).$$

5.4 Substitution preservation

The development defines simultaneous substitution:

$$\text{subst}(s, M),$$

where s maps de Bruijn indices to object-language terms. Under a binder, the substitution is lifted so that the newly bound variable maps to $\text{Var } 0$, while all substituted terms from the outer context are shifted.

The main theorem is:

$$\Gamma \vdash M : \tau \quad \text{and} \quad \forall n, \sigma \ (\text{lookup}(\Gamma, n) = \sigma \Rightarrow \Delta \vdash s(n) : \sigma) \quad \Rightarrow \quad \Delta \vdash \text{subst}(s, M) : \tau.$$

The single-variable substitution operation for beta contraction is:

$$\text{subst0}(N, M) = \text{subst}([0 \mapsto N, n + 1 \mapsto \text{Var}(n)], M).$$

The proved special case is:

$$\sigma, \Gamma \vdash M : \tau \quad \text{and} \quad \Gamma \vdash N : \sigma \quad \Rightarrow \quad \Gamma \vdash \text{subst0}(N, M) : \tau.$$

5.5 Beta preservation

One-step beta contraction is currently represented by the inductive relation:

$$\text{App}(\text{Lam}(\sigma, M), N) \rightarrow_\beta \text{subst0}(N, M).$$

The preservation theorem says:

$$M \rightarrow_\beta N \quad \text{and} \quad \Gamma \vdash M : \tau \quad \Rightarrow \quad \Gamma \vdash N : \tau.$$

5.6 The minimal logic H

The file `Bacon_Deduction.thy` now defines a Hilbert-style theoremhood judgment

$$\Gamma \vdash_H A,$$

read as: A is an H -theorem in the object-language type context Γ . It includes the Bacon-Dorr Figure 2 principles in the present syntax: classical truth-functional tautologies, universal instantiation, existential generalization, typed reflexivity, Leibniz's law, beta and eta conversion inside formula contexts, modus ponens, Hilbert-Ackermann generalization, and Hilbert-Ackermann instantiation.

The theory also defines a local-assumption wrapper

$$\Gamma; \Delta \vdash_H A,$$

where Δ is a list of object-language assumptions. At present this wraps assumptions, H -theorems, and modus ponens. The pure H judgment is the minimal inductive closure under

the Bacon-style rules.

The main checked metatheorem for this layer is formula preservation:

$$\Gamma \vdash_H A \quad \Rightarrow \quad \Gamma \vdash A : \text{Prop},$$

and similarly for the local-assumption judgment. This verifies that the derivability rules only derive well-typed object-language formulas.

5.7 Classicism

The file `Bacon_Abbreviations.thy` names the Boolean identities from Bacon-Dorr Figure 3 and the Classicist identities from Figure 4 as closed object-language formulas. The file `Bacon_Classicism.thy` then defines a second theoremhood judgment

$$\Gamma \vdash_C A,$$

read as: A is a theorem of Classicism in type context Γ . This judgment extends H with the six Boolean identities and the five Classicist identity schemas: the identity identity, absorption and distribution for universal quantification, and absorption and distribution for existential quantification.

The checked metatheorem for this layer is again formula preservation:

$$\Gamma \vdash_C A \quad \Rightarrow \quad \Gamma \vdash A : \text{Prop}.$$

There is also a local-assumption wrapper

$$\Gamma; \Delta \vdash_C A,$$

with the corresponding formula-preservation theorem.

5.8 Modal logicism

The file `Bacon_Modal.thy` defines broad necessity as propositional identity with object-language truth:

$$\Box_o A := A =_t \top_o.$$

It also defines possibility, entailment, and the usual modal schema terms:

$$\Diamond_o A := \sim \Box_o \sim A, \quad A \leq_o B := \Box_o (A \rightarrow B),$$

as well as object-language terms corresponding to the modal K , T , and 4 schemas. The theory proves typing lemmas for these definitions and adds small sanity checks, including that Classicism proves $\Box_o \top_o$, the conjunction commutativity identity, and the identity identity.

5.9 Restricted necessitation

The file `Bacon_Modal_Derivations.thy` proves the restricted necessitation principle for both H and C . In Bacon-style notation, the Classicism version is:

$$\Gamma \vdash_C A =_t \top_o \Rightarrow \Gamma \vdash_C \Box_o A.$$

There is an exactly analogous H version:

$$\Gamma \vdash_H A =_t \top_o \Rightarrow \Gamma \vdash_H \Box_o A.$$

These results are not yet the full necessitation theorem. They are immediate because broad necessity is defined as identity with truth:

$$\Box_o A := A =_t \top_o.$$

The corresponding Isabelle lemmas are:

```
lemma C_restricted_necessitation:
  assumes "Gamma \<turnstile>\<^sub>C Eq Prop A ObjTrue"
  shows "Gamma \<turnstile>\<^sub>C \<box>\<^sub>o A"

lemma H_restricted_necessitation:
  assumes "Gamma \<turnstile>\<^sub>H Eq Prop A ObjTrue"
  shows "Gamma \<turnstile>\<^sub>H \<box>\<^sub>o A"
```

Here `Eq Prop A ObjTrue` is the object-language formula $A =_t \top_o$. The source-level notation

`\<box>\<^sub>o A`

abbreviates `ObjBox A`.

5.10 Zero-ary equivalence closure

The same file also isolates the exact extra closure needed to move from “ A is equivalent to truth” to “ A is identical with truth.” The new judgment is written

$$\Gamma \vdash_{\text{CE}} A.$$

It extends C with the following propositional, zero-ary fragment of Bacon and Dorr’s Equivalence rule:

$$\frac{\Gamma \vdash A : t \quad \Gamma \vdash B : t \quad \Gamma \vdash_{\text{CE}} A \leftrightarrow B}{\Gamma \vdash_{\text{CE}} A =_t B}.$$

In Isabelle this rule is the constructor `CE_proves.PropEquivalence`:

`PropEquivalence`:

```
"Gamma \<turnstile> A : Prop ==>
Gamma \<turnstile> B : Prop ==>
Gamma \<turnstile>\<^sub>CE ObjIff A B ==>
Gamma \<turnstile>\<^sub>CE Eq Prop A B"
```

The extension also has modus ponens and the two Hilbert-Ackermann closure rules, and the file proves formula preservation:

$$\Gamma \vdash_{\text{CE}} A \Rightarrow \Gamma \vdash A : t.$$

With this rule available, necessitation follows once A has been connected to truth by object-language biconditional:

$$\Gamma \vdash A : t \text{ and } \Gamma \vdash_{\text{CE}} A \leftrightarrow \top_o \Rightarrow \Gamma \vdash_{\text{CE}} \Box_o A.$$

The Isabelle theorem name is

`CE_necessitation_from_equivalence_to_truth`.

The proof first uses `PropEquivalence` to infer $A =_t \top_o$, and then applies restricted necessitation.

5.11 Vector/zeta equivalence

The file `Bacon_Zeta.thy` now formalizes the higher-type vector Equivalence infrastructure. It introduces:

- `arrow_type`, the iterated arrow type $\sigma_1 \rightarrow \cdots \rightarrow \sigma_n \rightarrow \tau$;

- `app_vec`, iterated object-language application;
- `fresh_vars`, the de Bruijn variables `Var 0`, `Var 1`, ..., `Var (n-1)` in a freshly extended context;
- `shift_by`, which moves old terms across a finite fresh context prefix;
- `free_in` and `fresh_for_extension`, which state that the old shifted terms do not mention the newly added variables;
- `zeta_body`, the biconditional obtained by applying two shifted higher-type terms to the fresh variable vector.

The main typing theorem for this layer is:

$$\Gamma \vdash F : \vec{\sigma} \rightarrow t \quad \Rightarrow \quad \vec{\sigma}, \Gamma \vdash F^\dagger \vec{v} : t,$$

where $F^\dagger$ is the shifted version of F and $\vec{v}$ is the fresh de Bruijn variable vector. The corresponding Isabelle lemmas are `typed_app_fresh_vars` and `typed_zeta_body`.

The new derivability judgment is

$$\Gamma \vdash_{\text{CEV}} A.$$

It extends CE with the vector form of Bacon-Dorr Equivalence:

$$\frac{\vec{\sigma}, \Gamma \vdash_{\text{CEV}} F^\dagger \vec{v} \leftrightarrow G^\dagger \vec{v}}{\Gamma \vdash_{\text{CEV}} F =_{\vec{\sigma} \rightarrow t} G},$$

provided F and G are both typed as $\vec{\sigma} \rightarrow t$ in the old context Γ . The current theory also adds the contextual vector form needed for conditional higher-order extensionality:

$$\frac{\vec{\sigma}, \Gamma \vdash_{\text{CEV}} A^\dagger \rightarrow (F^\dagger \vec{v} \leftrightarrow G^\dagger \vec{v})}{\Gamma \vdash_{\text{CEV}} A \rightarrow F =_{\vec{\sigma} \rightarrow t} G}.$$

In Isabelle this is the `ContextVectorEquivalence` rule of `CEV_proves`. It is included in formula preservation, `CEV_theorem_to_truth_induction`, semantic soundness, and the canonical `CEV_rule_closed` bookkeeping through `CEV_context_vector_equivalence_closed`. Formula preservation has been proved:

$$\Gamma \vdash_{\text{CEV}} A \quad \Rightarrow \quad \Gamma \vdash A : t.$$

The zero-vector instance recovers the propositional rule:

$$\Gamma \vdash A : t, \quad \Gamma \vdash B : t, \quad \Gamma \vdash_{\text{CEV}} A \leftrightarrow B \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} A =_t B.$$

Consequently, the previous restricted-necessitation bridge also holds for CEV:

$$\Gamma \vdash A : t, \quad \Gamma \vdash_{\text{CEV}} A \leftrightarrow \top_o \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} \Box_o A.$$

5.12 Theoremhood as equivalence with truth

The same file now proves the theorem-to-truth equivalence step for CEV. First, it proves that object-language truth is itself an H -theorem, hence a CEV-theorem:

$$\Gamma \vdash_H \top_o \quad \text{and} \quad \Gamma \vdash_{\text{CEV}} \top_o.$$

Then it proves a general propositional closure lemma:

$$\Gamma \vdash_{\text{CEV}} A \quad \text{and} \quad \Gamma \vdash_{\text{CEV}} B \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} A \leftrightarrow B.$$

The theorem-to-truth result follows by taking B to be $\top_o$:

$$\Gamma \vdash_{\text{CEV}} A \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} A \leftrightarrow \top_o.$$

The Isabelle theorem names are:

`CEV_theorem_equiv_ObjTrue` `CEV_theorem_to_truth_induction`.

The second theorem is stated and checked as an induction over the constructors of

`CEV_proves`.

In each case, the relevant CEV theorem is reconstructed from the rule premises and then converted to equivalence with truth.

As a corollary, the development now proves unrestricted CEV necessitation:

$$\Gamma \vdash_{\text{CEV}} A \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} \Box_o A.$$

The Isabelle theorem name is `CEV_necessitation`.

5.13 Standalone beta-eta conversion

The file `Bacon_Conversion.thy` now factors beta-eta conversion itself away from the proof systems. In Bacon-style notation, write the new relation as

$$\Gamma \vdash M \equiv_{\beta\eta, \tau} N.$$

The subscript τ is important: this is a typed syntactic conversion relation, not an untyped rewrite relation and not a quotient of Isabelle terms. It is generated by five clauses:

$$\frac{\Gamma \vdash M : \tau}{\Gamma \vdash M \equiv_{\beta\eta,\tau} M'}, \quad \frac{\Gamma \vdash M : \tau \quad \Gamma \vdash N : \tau \quad M \rightarrow_{\beta} N}{\Gamma \vdash M \equiv_{\beta\eta,\tau} N},$$

$$\frac{\Gamma \vdash M : \tau \quad \Gamma \vdash N : \tau \quad M \rightarrow_{\eta} N}{\Gamma \vdash M \equiv_{\beta\eta,\tau} N}, \quad \frac{\Gamma \vdash M \equiv_{\beta\eta,\tau} N}{\Gamma \vdash N \equiv_{\beta\eta,\tau} M},$$

$$\frac{\Gamma \vdash M \equiv_{\beta\eta,\tau} N \quad \Gamma \vdash N \equiv_{\beta\eta,\tau} P}{\Gamma \vdash M \equiv_{\beta\eta,\tau} P}.$$

Here $M \rightarrow_{\beta} N$ abbreviates a compatible one-step beta contraction, and $M \rightarrow_{\eta} N$ abbreviates a compatible one-step eta contraction.

The Isabelle relation is:

```
inductive beta_eta_equiv ::
  "ctx => otype => oterm => oterm => bool" where
  Refl: "Gamma \<turnstile> M : tau ==> beta_eta_equiv Gamma tau M
    \ M"
| Beta: "Gamma \<turnstile> M : tau ==> Gamma \<turnstile> N : tau
  \ ==>
    compatible_step beta_contract M N ==>
    beta_eta_equiv Gamma tau M N"
| Eta: "Gamma \<turnstile> M : tau ==> Gamma \<turnstile> N : tau
  \ ==>
    compatible_step eta_contract M N ==>
    beta_eta_equiv Gamma tau M N"
| Sym: "beta_eta_equiv Gamma tau M N ==>
  beta_eta_equiv Gamma tau N M"
| Trans: "beta_eta_equiv Gamma tau M N ==>
  beta_eta_equiv Gamma tau N P ==>
  beta_eta_equiv Gamma tau M P"
```

The first checked metatheorem for the relation is type preservation:

$$\Gamma \vdash M \equiv_{\beta\eta,\tau} N \quad \Rightarrow \quad \Gamma \vdash M : \tau \text{ and } \Gamma \vdash N : \tau.$$

In Isabelle this is `beta_eta_equiv_types`. The one-sided corollaries are:

`beta_eta_equiv_left_type`
`beta_eta_equiv_right_type.`

The second checked metatheorem is the bridge back to derivability. At proposition type, beta-eta equivalence yields an object-language biconditional:

$$\Gamma \vdash A \equiv_{\beta\eta,t} B \quad \Rightarrow \quad \Gamma \vdash_H A \leftrightarrow B.$$

This is `H_beta_eta_equiv`. Its proof is an induction on the conversion derivation. The beta and eta cases use the existing *H* conversion axioms. The reflexive, symmetric, and transitive cases use small propositional tautology lemmas for biconditional reflexivity, symmetry, and transitivity.

The same conversion-to-biconditional bridge is lifted through the stronger systems:

```
lemma C_beta_eta_equiv:
  assumes "beta_eta_equiv Gamma Prop A B"
  shows "Gamma \<turnstile>\<^sub>C (A
    - \<longleftarrow>\<^sub>o B)"
```

```
lemma CE_beta_eta_equiv:
  assumes "beta_eta_equiv Gamma Prop A B"
  shows "Gamma \<turnstile>\<^sub>CE (A
    - \<longleftarrow>\<^sub>o B)"
```

```
lemma CEV_beta_eta_equiv:
  assumes "beta_eta_equiv Gamma Prop A B"
  shows "Gamma \<turnstile>\<^sub>CEV (A
    - \<longleftarrow>\<^sub>o B)"
```

This gives later proofs a clean conversion interface: they can reason about the standalone beta-eta closure, then import the corresponding biconditional theorem only when a particular proof system is needed.

5.14 Local CEV derivability

The file `Bacon_S4.thy` adds a deliberately small local derivability relation for working under one temporary assumption. In Bacon-style notation, write it as

$$\Gamma; A \vdash_{\text{CEV}}^{\text{loc}} B.$$

It has only three constructors:

$$\frac{\Gamma \vdash A : t}{\Gamma; A \vdash_{\text{CEV}}^{\text{loc}} A}, \quad \frac{\Gamma \vdash_{\text{CEV}} B}{\Gamma; A \vdash_{\text{CEV}}^{\text{loc}} B}, \quad \frac{\Gamma; A \vdash_{\text{CEV}}^{\text{loc}} B \quad \Gamma; A \vdash_{\text{CEV}}^{\text{loc}} B \rightarrow C}{\Gamma; A \vdash_{\text{CEV}}^{\text{loc}} C}.$$

The Isabelle relation is:

```
inductive CEV_from :: "ctx => oterm => oterm => bool" where
  Assumption: "Gamma \<turnstile> A : Prop ==> CEV_from Gamma A A"
| Theorem: "Gamma \<turnstile>\<^sub>CEV B ==> CEV_from Gamma A B"
| MP: "CEV_from Gamma A B ==>
      CEV_from Gamma A (Imp B C) ==>
      CEV_from Gamma A C"
```

The key metatheorem is the corresponding deduction theorem:

$$\Gamma; A \vdash_{\text{CEV}}^{\text{loc}} B \quad \text{and} \quad \Gamma \vdash A : t \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} A \rightarrow B.$$

In Isabelle this is `CEV_from_deduction`. This relation is not a new object-language axiom. It is just proof bookkeeping for deriving conditional theorems from temporary assumptions.

5.15 Equality symmetry and shifted beta reduction

The same file proves the small substitution fact needed to beta-reduce lambda terms whose body contains an old term shifted across a fresh binder:

$$\text{subst0}(N, \text{shift}(M)) = M.$$

In Isabelle this appears as `subst0_shift` and its unfolded variant `subst_case_nat_shift`. This is the de Bruijn version of the familiar fact that substituting for the newly bound variable does not affect an older free term once the bookkeeping shift is undone.

Using reflexivity, Leibniz's Law, and beta conversion, the theory then derives symmetry of object-language identity:

$$\Gamma \vdash M : \sigma, \quad \Gamma \vdash N : \sigma \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} (M =_{\sigma} N) \rightarrow (N =_{\sigma} M).$$

The Bacon-style proof is standard. Define

$$F(x) := x =_{\sigma} M.$$

Reflexivity gives $M =_{\sigma} M$. Leibniz's Law gives

$$M =_{\sigma} N \rightarrow (F(M) \rightarrow F(N)).$$

But $F(M)$ beta-reduces to $M =_\sigma M$, and $F(N)$ beta-reduces to $N =_\sigma M$. A local derivation under the assumption $M =_\sigma N$, followed by `CEV_from_deduction`, yields the displayed conditional. The Isabelle theorem name is `CEV_eq_sym`.

5.16 The modal T schema

The first S4-style modal theorem now proved is T :

$$\Gamma \vdash A : t \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} \Box_o A \rightarrow A.$$

Equivalently, since `modal_T A` is defined as `Imp (ObjBox A) A`, the Isabelle theorem is:

`lemma CEV_modal_T:`

```
  assumes "Gamma \<turnstile> A : Prop"
  shows "Gamma \<turnstile>\<^sub>CEV modal_T A"
```

The proof uses the definition $\Box_o A := A =_t \top_o$. From $\Box_o A$, equality symmetry gives $\top_o =_t A$. Then apply Leibniz's Law to the propositional identity function

$$I_t := \lambda p^t. p.$$

Leibniz gives

$$\top_o =_t A \rightarrow (I_t(\top_o) \rightarrow I_t(A)).$$

The theory proves $I_t(\top_o)$, because $\top_o$ is already a CEV-theorem and $I_t(\top_o)$ beta-expands/contracts with it. It then derives $I_t(A)$, and beta conversion yields A . Finally, the local assumption $\Box_o A$ is discharged by `CEV_from_deduction`.

5.17 Equality transfer for K and 4

The next step was to avoid adding a new local Equivalence rule. Instead, the formalization derives the identity-transfer facts needed for K and 4 from Leibniz's Law, beta conversion, global necessitation, and ordinary local deduction.

First, `Bacon_S4.thy` proves equality transitivity:

$$\Gamma \vdash M : \sigma, \quad \Gamma \vdash N : \sigma, \quad \Gamma \vdash P : \sigma \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} (M =_\sigma N) \rightarrow ((N =_\sigma P) \rightarrow (M =_\sigma P)).$$

The Isabelle theorem is `CEV_eq_trans`. Its proof again uses a local derivation under the conjunction of the two equality assumptions, followed by deduction and currying.

Second, the theory proves that $\top_o \rightarrow B$ is propositionally identical to B :

$$\Gamma \vdash B : t \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} (\top_o \rightarrow B) =_t B.$$

This is `CEV_eq_imp_ObjTrue_left`. The proof first derives the biconditional $(\top_o \rightarrow B) \leftrightarrow B$, using the theoremhood of $\top_o$, and then applies zero-ary vector Equivalence.

Third, the theory proves the key equality-to-truth transfer:

$$\Gamma \vdash M : \sigma, \quad \Gamma \vdash N : \sigma \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} (M =_\sigma N) \rightarrow ((M =_\sigma N) =_t \top_o).$$

The Isabelle theorem is `CEV_eq_truth_of_eq`. The idea is simple: reflexivity gives $M =_\sigma M$, unrestricted necessitation gives $(M =_\sigma M) =_t \top_o$, and Leibniz transfers this fact along $M =_\sigma N$ using the predicate

$$x \mapsto (M =_\sigma x) =_t \top_o.$$

5.18 The modal K and 4 schemata

The modal K theorem is now:

$$\Gamma \vdash A : t, \quad \Gamma \vdash B : t \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} \Box_o(A \rightarrow B) \rightarrow (\Box_o A \rightarrow \Box_o B).$$

In Isabelle:

`lemma CEV_modal_K:`

```
  assumes "Gamma \<turnstile> A : Prop"
    and "Gamma \<turnstile> B : Prop"
  shows "Gamma \<turnstile>\<^sub>CEV modal_K A B"
```

The proof reasons under the conjunction of the two boxed assumptions

$$(A \rightarrow B) =_t \top_o \quad \text{and} \quad A =_t \top_o$$

and derives $B =_t \top_o$. The main move is to apply Leibniz's Law to

$$p \mapsto ((p \rightarrow B) =_t \top_o).$$

This changes $(A \rightarrow B) =_t \top_o$ into $(\top_o \rightarrow B) =_t \top_o$. Since $\top_o \rightarrow B$ is identical to B , equality symmetry and transitivity yield $B =_t \top_o$. Deduction and currying then produce the displayed K theorem.

The modal 4 theorem is:

$$\Gamma \vdash A : t \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} \Box_o A \rightarrow \Box_o \Box_o A.$$

In Isabelle:

```
lemma CEV_modal_4:
  assumes "Gamma \<turnstile> A : Prop"
  shows "Gamma \<turnstile>\<^sub>CEV modal_4 A"
```

This is exactly the proposition-type instance of `CEV_eq_truth_of_eq`: under the assumption $A =_t \top_o$, derive

$$(A =_t \top_o) =_t \top_o.$$

Together with `CEV_modal_T`, these lemmas give the expected S4-style package for the object-language box defined as identity with truth.

The theory also records the package theorem:

```
theorem CEV_S4_package:
  assumes "Gamma \<turnstile> A : Prop"
  and "Gamma \<turnstile> B : Prop"
  shows "Gamma \<turnstile>\<^sub>CEV modal_K A B"
  and "Gamma \<turnstile>\<^sub>CEV modal_T A"
  and "Gamma \<turnstile>\<^sub>CEV modal_4 A"
  and "!!C. Gamma \<turnstile>\<^sub>CEV C ==> Gamma
    \<turnstile>\<^sub>CEV \<box>\<^sub>o C"
```

6 Initial semantic layer

The file `Bacon_Semantics.thy` begins the semantic side of the formalization. It deliberately does not yet construct a concrete set-theoretic model. Instead, it introduces an abstract applicative structure. In Bacon-style notation, such a structure supplies:

$$D_\sigma, \llbracket c : \sigma \rrbracket, \text{app}, \text{lam}_\sigma, \text{true}(b), \text{holds}, =_\sigma^M.$$

Here D_σ is the semantic domain for object type σ ; `app` and `lam` interpret object-language application and abstraction; `true(b)` embeds a metalanguage Boolean as an object-language proposition-denotation; `holds` decodes proposition-denotations; and $=_\sigma^M$ is the type-indexed semantic interpretation of object-language identity.

The Isabelle locale is:

```

locale applicative_structure =
  fixes D :: "otype => 'v set"
  and const_den :: "string => otype => 'v"
  and app_den :: "'v => 'v => 'v"
  and lam_den :: "otype => ('v => 'v) => 'v"
  and truth_den :: "bool => 'v"
  and holds :: "'v => bool"
  and eq_den :: "otype => 'v => 'v => bool"

```

with assumptions saying that constants have their displayed type, application preserves arrow-domain typing, abstraction forms arrow-domain elements, truth values denote propositions, extensionally equal abstraction bodies have equal lambda denotations, beta and eta are semantically respected by

$$\text{app}(\text{lam}_\sigma F, x) = F(x), \quad \text{lam}_\sigma(\lambda x. \text{app}(f, x)) = f,$$

semantic identity is reflexive on each domain, and Leibniz substitution is sound for proposition-valued predicates:

$$x =_\sigma^M y \Rightarrow F(x) \text{ true} \Rightarrow F(y) \text{ true}.$$

Semantic environments are de Bruijn-indexed functions. The notation

$$\rho \models \Gamma$$

corresponds to the Isabelle predicate `env_typed Gamma rho`; it says that whenever

$$\text{lookup Gamma } n = \text{Some sigma},$$

the value $\rho(n)$ belongs to D_σ . The operation `extend_env x rho` is the semantic counterpart of moving under one binder: index 0 is assigned x , and index $n + 1$ looks back to $\rho(n)$.

The recursive interpretation function is `eval`. In standard notation, write it as $\llbracket M \rrbracket_\rho$. The central checked theorem is typed evaluation:

$$\Gamma \vdash M : \tau \quad \text{and} \quad \rho \models \Gamma \quad \Rightarrow \quad \llbracket M \rrbracket_\rho \in D_\tau.$$

In Isabelle:

```
lemma eval_type:
```

```

assumes "Gamma \<turnstile> M : tau"
  and "env_typed Gamma rho"
shows "eval rho M \<in> D tau"

```

The semantic layer now also proves that the syntactic infrastructure commutes with interpretation. Renaming and substitution are evaluated by the corresponding semantic environments:

```

lemma eval_rename:
  "eval rho (rename r M) = eval (%n. rho (r n)) M"

```

```

lemma eval_subst0:
  "eval rho (subst0 T M) = eval (extend_env (eval rho T) rho) M"

```

Beta and eta contractions are then shown to preserve denotation, and that fact is lifted through all compatible formula contexts:

```

lemma beta_contract_eval_preserving:
  "eval_preserving_step beta_contract"

```

```

lemma eta_contract_eval_preserving:
  "eval_preserving_step eta_contract"

```

```

lemma compatible_step_eval:
  assumes "compatible_step R M N"
    and "eval_preserving_step R"
    and "Gamma \<turnstile> M : tau"
    and "Gamma \<turnstile> N : tau"
    and "env_typed Gamma rho"
  shows "eval rho M = eval rho N"

```

The file also connects the existing syntactic propositional-tautology predicate to semantic truth. First it proves that semantic truth for the displayed truth-functional connectives agrees with `prop_eval`:

```

lemma holds_eval_prop_eval:
  "holds (eval rho A) = prop_eval (%B. holds (eval rho B)) A"

```

Then it proves soundness for propositional tautologies:

$$\text{prop_tautology}(\Gamma, A) \text{ and } \rho \models \Gamma \Rightarrow \text{holds}(\llbracket A \rrbracket_\rho).$$

The corresponding Isabelle theorem is `prop_tautology_sound`.

Finally, `Bacon_Semantics.thy` defines a semantic validity predicate:

$$\Gamma \models A \text{ iff } \Gamma \vdash A : t \text{ and every } \rho \models \Gamma \text{ makes } A \text{ true.}$$

In Isabelle this is `valid_in_context`. Propositional tautologies remain available as a checked validity result:

```
lemma prop_tautology_valid:
  assumes "prop_tautology Gamma A"
  shows "valid_in_context Gamma A"
```

The new milestone is full soundness for the minimal higher-order logic H :

$$\Gamma \vdash_H A \Rightarrow \Gamma \models A.$$

In Isabelle:

```
theorem H_soundness:
  assumes "Gamma \<turnstile>\<^sub>H A"
  shows "valid_in_context Gamma A"
```

The induction covers propositional tautologies, universal instantiation, existential generalization, typed reflexivity, Leibniz's Law, beta/eta conversion in formula contexts, modus ponens, and the Hilbert-Ackermann generalization and instantiation rules.

The stronger proof systems require stronger semantic hypotheses. The file therefore introduces three locale extensions. A `classicist_structure` is an applicative structure in which the Boolean identities and the five Classicist identity schemata are valid. In that locale:

```
theorem C_soundness:
  assumes "Gamma \<turnstile>\<^sub>C A"
  shows "valid_in_context Gamma A"
```

A `propositional_equivalence_structure` adds propositional extensionality: if two proposition denotations have the same truth value, then they are identified by `eq_den Prop`. This validates the zero-ary Equivalence rule:

```
theorem CE_soundness:
  assumes "Gamma \<turnstile>\<^sub>CE A"
  shows "valid_in_context Gamma A"
```

Finally, a `vector_equivalence_structure` adds the corresponding vector extensionality condition: if two proposition-valued functions agree in truth on every typed vector of arguments, then they are identical at the appropriate arrow type. The semantic proof uses `extend_envs`, `eval_shift_by_extend_envs`, and `eval_app_vec` to relate the fresh de Bruijn vector in `zeta_body` to ordinary semantic argument vectors. In that locale:

```
theorem CEV_soundness:
  assumes "Gamma \<turnstile>\<^sub>CEV A"
  shows "valid_in_context Gamma A"
```

7 Completeness interface

The file `Bacon_Completeness.thy` begins the completeness side without yet constructing a canonical model. It defines semantic satisfaction for a finite list of assumptions:

```
definition satisfies_assumptions ::
  "ctx => oterm list => 'v env => bool"
```

and semantic consequence:

```
definition semantically_entails ::
  "ctx => oterm list => oterm => bool"
```

In Bacon-style notation, $\Gamma; \Delta \models_s A$ means that A is a well-typed formula and every environment satisfying all members of Δ makes A true.

The file then proves local soundness for the two derivability systems already defined with explicit assumption lists:

```
lemma H_derivable_sound:
  assumes "Gamma ; Delta \<turnstile>\<^sub>H A"
  shows "Gamma ; Delta \<Turnstile>\<^sub>s A"
```

```
lemma C_derivable_sound:
  assumes "Gamma ; Delta \<turnstile>\<^sub>C A"
  shows "Gamma ; Delta \<Turnstile>\<^sub>s A"
```

Finally, the file isolates the exact countermodel principles that a canonical model construction must supply. For example:

```
definition H_countermodel_property :: bool
```

```

theorem H_completeness_from_countermodels:
  assumes "H_countermodel_property"
    and "valid_in_context Gamma A"
  shows "Gamma \<turnstile>\<^sub>H A"

theorem H_strong_completeness_from_countermodels:
  assumes "H_ entailment_countermodel_property"
    and "Gamma ; Delta \<Turnstile>\<^sub>s A"
  shows "Gamma ; Delta \<turnstile>\<^sub>H A"

```

Analogous bridge theorems are checked for C , CE , and CEV . Thus the remaining completeness task is now sharply formulated: construct, for each unprovable well-typed formula, a semantic countermodel in the appropriate semantic locale.

8 Canonical theory layer

The file `Bacon_Canonical.thy` now carries most of the syntactic Henkin construction. In standard notation, it replaces finite assumption lists Δ by sets T of formulas and defines:

$$\Gamma; T \vdash_H^s A \quad \text{iff} \quad \text{some finite list } \Delta \subseteq T \text{ satisfies } \Gamma; \Delta \vdash_H A.$$

The analogous relation $\Gamma; T \vdash_C^s A$ is defined for C .

The file then defines typed theories, consistency, negation-completeness, maximal consistency, and Henkin witnessing. The Henkin condition is:

$$\text{if } \exists x^\sigma A(x) \in T, \text{ then for some term } W : \sigma, A(W) \in T.$$

In Isabelle this is represented as membership of `subst0 W A` whenever `Exists sigma A` belongs to T .

The main verified facts now go well beyond the initial closure lemmas. The file proves Lindenbaum-Henkin extension theorems for H and C , using countability-based enumerations of formulas and witness bodies. It proves the canonical truth clauses for negation, conjunction, implication, disjunction, universal and existential quantification, equality, Leibniz substitution, and beta-eta conversion. It then factors the truth lemma through substitutional truth predicates and a term-model semantics: a term-model value is a typed object-language term, evaluation is capture-avoiding substitution, and propositional truth is membership in the Henkin theory. This gives verified H/C Henkin-validity and term-model completeness

biconditionals, plus explicit H/C term-model countermodels for unprovable formulas and underivable finite assumption entailments.

9 CE and CEV completeness infrastructure

The current canonical file also isolates the extra closure needed above Classicism. A CE-Henkin theory is a *C*-Henkin theory plus closure under propositional Equivalence and the CE generalization/instantiation rules. A CEV-Henkin theory adds vector Equivalence and the CEV rule closures. The verified containment lemmas show that every CE or CEV theorem belongs to every corresponding Henkin theory.

The construction then introduces strengthened local consequence relations. For CE, `CE_equiv_set_derivable` internalizes the local rule from $A \leftrightarrow B$ to $A =_t B$. The later `CE_context_equiv_set_derivable` relation strengthens this further: from a locally derivable finite-prefix implication

$$P_1 \rightarrow \dots \rightarrow P_n \rightarrow (A \leftrightarrow B)$$

it locally derives

$$P_1 \rightarrow \dots \rightarrow P_n \rightarrow (A =_t B).$$

This isolates exactly the prefix-Equivalence closure needed for CE completeness. The file proves the associated monotonicity, finite support, deduction, Lindenbaum, Henkin-witness, and shifted-instantiation lemmas, and reduces CE completeness to the corresponding prefix internalization principle.

For CEV, the analogous strengthened relation `CEV_context_equiv_set_derivable` has now been developed. The verified package includes monotonicity, a deduction theorem, finite support, consistency-preserving one-step extensions, a Lindenbaum extension theorem, prefix/conservativity equivalences, and conditional Henkin witness staging. The shifted existential-instantiation principle for this relation is no longer an external assumption: it is proved from CEV-context conservativity. The remaining named CEV construction obligation is

$$\text{CEV_context_equiv_abstract_const_admissible.}$$

Informally, this says that if A is locally derivable from T , then after abstracting a fresh constant $c : \sigma$, the abstracted formula is locally derivable from the abstracted theory in the extended context. This is the fresh-constant abstraction step needed to preserve consistency

when Henkin witness axioms are added. It is expected to require the renaming/substitution behavior of CEV theoremhood through `zeta_body` and vector Equivalence.

With this abstraction principle available, the file already proves:

`CEV_context_equiv_available_countermodel_property`

from CEV prefix internalization and abstraction. The existing bridge theorems then turn that property into the corresponding CEV Henkin-validity and term-model completeness statements.

10 Caie's names layer

The file `Bacon_Caie.thy` is the first application layer over the Bacon/Bacon-Dorr background logic. It does not replace the deep embedding by a shallow HOL formalization. Instead, Caie's vocabulary is represented as object-language syntax inside the same typed object language used for H , C , CE , and CEV .

10.1 Caie-style types

The type correspondence is:

Caie notation	Isabelle definition	Report notation
e	<code>Ind</code>	<code>Ind</code>
t	<code>Prop</code>	<code>Prop</code>
$e \rightarrow t$	<code>caie_prop_ty</code>	<code>Pr</code>
$(e \rightarrow t) \rightarrow t$	<code>caie_classifier_ty</code>	<code>Cl</code>

Thus an ordinary first-order property is an object-language predicate of individuals, and a name meaning is represented as a property classifier:

$$\text{Pr} := \text{Ind} \Rightarrow_o \text{Prop}, \quad \text{Cl} := \text{Pr} \Rightarrow_o \text{Prop}.$$

The theory also defines the corresponding class types $\text{Pr} \Rightarrow_o \text{Prop}$ and $\text{Cl} \Rightarrow_o \text{Prop}$, which are used by the maximal consistency vocabulary.

Caie's counterfactual connective is currently an uninterpreted object-language constant:

$$A \Box \rightarrow_o B.$$

In Isabelle this is `ObjCF A B`. Its source-level infix notation is:

`\<box>\<rightarrow>\<^sub>o`

The connective itself remains uninterpreted in the definitional layer. The Caie-specific counterfactual principles are now collected separately in the local axiom package described below, so the definitions stay conservative over the Bacon object logic.

10.2 Basic name-theoretic operations

The main definitional operations now represented are:

$$\begin{aligned} \text{heq}(x) &:= \lambda y^e. y =_e x, \\ Q^{\downarrow c} &:= \lambda x^e. Q(\text{heq}(x)), \\ P^{\downarrow c}(R) &:= (\exists x^e P =_{\text{Pr}} \text{heq}(x)) \Box \rightarrow_o (\exists x^e (P(x) \& R(x))), \\ Q \sim_{\downarrow c} Z &:= Q^{\downarrow c} =_{\text{Pr}} Z^{\downarrow c}. \end{aligned}$$

The Isabelle definitions use de Bruijn indices, so the displayed equations are the intended Bacon/Caie reading rather than the literal source syntax. For example, the body of $P^{\downarrow c}(R)$ is named `caie_up_body P R`; inside binders, the Isabelle source shifts P and R before applying them to `Var 0`. This is the usual de Bruijn bookkeeping: the shift says that P and R are free relative to the newly bound individual variable.

The file also defines the predicate vocabulary used in Caie’s Appendix C:

GLB, NS, UC, GLBC, Comp, Cons, MC,
Hae, hae, NH, AH, AC, HP, NHP, ACF, PE,
Name, WName, phae.

These are object-language constants or lambda-defined operations, not Isabelle predicates. Applied abbreviations such as `caie_Name Q` and `caie_MC Q` merely build object-language formulas.

10.3 Typing facts

The theory proves typing lemmas for the primitive and applied Caie vocabulary. For example:

$$\begin{aligned} \Gamma \vdash x : \text{Ind} &\Rightarrow \Gamma \vdash \text{heq}(x) : \text{Pr}, \\ \Gamma \vdash Q : \text{Cl} &\Rightarrow \Gamma \vdash Q^{\downarrow c} : \text{Pr}, \end{aligned}$$

$$\Gamma \vdash P : \text{Pr} \Rightarrow \Gamma \vdash P^{\downarrow c} : \text{Cl},$$

and

$$\Gamma \vdash Q : \text{Cl}, \quad \Gamma \vdash Z : \text{Cl} \Rightarrow \Gamma \vdash Q \sim_{\downarrow c} Z : \text{Prop}.$$

There are also applied typing lemmas for the Appendix C vocabulary, including `typed_caie_MC`, `typed_caie_Hae`, `typed_caie_Name`, and `typed_caie_WName`. Finally, the deep-embedded Appendix C statements `caie_Thm32` through `caie_Thm40` are all proved to be formulas:

$$A \in \text{set } \text{caie_appendix_C_theorems} \Rightarrow \Gamma \vdash A : \text{Prop}.$$

10.4 Definitional conversion bridges

The main verified Caie facts are beta-eta conversion bridges. The standalone conversion relation proves the definitional reductions first; the general bridge lemmas then turn these into object-language biconditionals in the proof systems. The currently named Caie bridges are:

$$\begin{aligned} \text{heq}(x)(y) &\simeq_{\beta\eta} y =_e x, \\ Q^{\downarrow c}(x) &\simeq_{\beta\eta} Q(\text{heq}(x)), \\ P^{\downarrow c}(R) &\simeq_{\beta\eta} \text{caie_up_body } P \ R, \\ Q \sim_{\downarrow c} Z &\simeq_{\beta\eta} Q^{\downarrow c} =_{\text{Pr}} Z^{\downarrow c}. \end{aligned}$$

The corresponding Isabelle theorem names are:

```
beta_eta_caie_heq_apply,  H_caie_heq_apply, CEV_caie_heq_apply,
beta_eta_caie_down_apply, H_caie_down_apply, CEV_caie_down_apply,
beta_eta_caie_up_apply,   H_caie_up_apply,  CEV_caie_up_apply,
beta_eta_caie_dsim,       H_caie_dsim,      CEV_caie_dsim.
```

For instance, if $Q : \text{Cl}$ and $x : e$, then Isabelle verifies:

$$\Gamma \vdash_H (Q^{\downarrow c}(x) \leftrightarrow Q(\text{heq}(x)))$$

and also the analogous CEV theorem. The same pattern holds for `heq`, `·↓c`, and `~↓c`.

The proof of the `·↓c` and `~↓c` bridges required a small amount of de Bruijn substitution infrastructure. These lemmas are deliberately local and mostly used explicitly rather than installed as global simplifier rules. That keeps later proof search predictable.

10.5 Local Caie derivability

The latest Caie step adds an explicit local derivability interface for working from Caie-specific assumptions without baking those assumptions into the base logic. In standard notation, write

$$\Gamma; \Delta \vdash_{\text{Caie}} A$$

for derivability of A from a finite list Δ of object-language assumptions over the background context Γ . The Isabelle predicate is:

`caie_CEV_derivable` `Gamma` `Delta` `A`

It has exactly three primitive clauses:

$$\frac{A \in \Delta \quad \Gamma \vdash A : \text{Prop}}{\Gamma; \Delta \vdash_{\text{Caie}} A}, \quad \frac{\Gamma \vdash_{\text{CEV}} A}{\Gamma; \Delta \vdash_{\text{Caie}} A},$$

and modus ponens:

$$\frac{\Gamma; \Delta \vdash_{\text{Caie}} A \quad \Gamma; \Delta \vdash_{\text{Caie}} A \rightarrow B}{\Gamma; \Delta \vdash_{\text{Caie}} B}.$$

Thus $\vdash_{\text{Caie}}$ is not a new logic. It is a bookkeeping layer over CEV theoremhood plus explicitly listed Caie assumptions.

The verified metatheory for this wrapper includes:

$$\Gamma; \Delta \vdash_{\text{Caie}} A \quad \Rightarrow \quad \Gamma \vdash A : \text{Prop},$$

and monotonicity in the assumption list:

$$\Gamma; \Delta \vdash_{\text{Caie}} A, \quad \Delta \subseteq \Delta' \quad \Rightarrow \quad \Gamma; \Delta' \vdash_{\text{Caie}} A.$$

There is also a bridge back to the existing one-assumption machinery in `Bacon_S4.thy`. If A is well-typed and B is derivable from the singleton assumption list $[A]$, then:

$$\Gamma; [A] \vdash_{\text{Caie}} B \quad \Rightarrow \quad \Gamma \vdash_{\text{CEV}} A \rightarrow B.$$

In Isabelle this is `caie_CEV_derivable_singleton_deduction`, proved by first translating the singleton derivation into `CEV_from` and then using `CEV_from_deduction`.

The four definitional conversion bridges have also been lifted into arbitrary Caie assumption contexts. For example, if $Q : \text{Cl}$ and $x : e$, then:

$$\Gamma; \Delta \vdash_{\text{Caie}} (Q^{\text{!c}}(x) \leftrightarrow Q(\text{heq}(x))).$$

The corresponding theorem names are:

```
caie_CEV_derivable_heq_apply, caie_CEV_derivable_down_apply,
caie_CEV_derivable_up_apply,  caie_CEV_derivable_dsim.
```

The wrapper also has derivability-level eliminators for object-language biconditionals:

```
caie_CEV_derivable_bicond_left,
caie_CEV_derivable_bicond_right.
```

These rules let a local proof use a derivable definitional bridge $A \leftrightarrow B$ together with one side of the biconditional to derive the other side without leaving the $\vdash_{\text{Caie}}$ wrapper. This means that later derivations from Caie’s substantive name and counterfactual assumptions can use the beta-eta reductions without reopening their conversion proofs.

Finally, the file defines a typed assumption-package predicate:

```
caie_assumption_package  $\Gamma \Delta$  iff every member of  $\Delta$  is a  $\Gamma$ -formula.
```

The Appendix C target list is proved to be such a package. Consequently, members of `caie_appendix_C_theorems` are locally available when that list is used explicitly as an assumption list. This is only bookkeeping: it does not turn Appendix C statements into theorems of CEV.

10.6 Caie-specific axiom package

The file now defines an explicit, non-circular package of Caie-style principles for the residual Appendix C work. These formulas are not the Appendix C target theorems themselves. They are lower-level assumptions corresponding to the counterfactual and name/haecceity principles cited in Caie’s proof dependencies.

The counterfactual part is:

```
caie_counterfactual_principles.
```

It contains closed proposition-level versions of counterfactual identity, reciprocity, modus ponens, conditional excluded middle, non-triviality, absorption,

$$(p \Box \rightarrow_o q) \leftrightarrow (p \Box \rightarrow_o (p \Box \rightarrow_o q)),$$

and consequent strengthening,

$$\Box(q \rightarrow r) \rightarrow ((p \Box_{\rightarrow_o} q) \rightarrow (p \Box_{\rightarrow_o} r)).$$

The name-theoretic part is:

`caie_name_haecceity_principles.`

It contains lower-level principles for possible haecceity projection, name down-naturality, a component-box principle for the down-projection possible haecceity conditions, modal collapse for possible haecceities, the five component conditions needed to make $P^{\dagger_c}$ a Name when P is a possible haecceity, pointwise up-down and down-up principles for possible haecceities and names, the haecceity case of $P^{\dagger_c}$, a possible-haecceity witness principle for the unfolded upward-projection body, weak-name possible-haecceity principles, weak-name haecceity classification, the Hae down-characterization, and weak-name down-similarity extensionality. The component-box principle is named

`caie_name_expanded_down_phae_components_principle.`

It is deliberately weaker than the previously used `caie_name_expanded_down_phae_principle`: it gives the boxed possible, positive-persistence, and negative-persistence components separately, rather than assuming the packed `phae` formula.

The combined package is `caie_appendix_C_axiom_package`. Its definition appends the counterfactual-principle list to the name/haecceity-principle list. Isabelle verifies that every member is a closed object-language formula:

`caie_appendix_C_axiom_package_typed.`

It also verifies that any member of this package is locally derivable when the package itself is used as the local assumption list. The corresponding theorem is:

`caie_CEV_derivable_appendix_C_axiom.`

At the package boundary, the file names the closure predicate:

`caie_appendix_C_axiom_package_closes_targets` Γ .

By definition, this says that every member of the current principle-target bucket is derivable

from the new axiom package. The destructor is:

`caie_appendix_C_axiom_package_closes_targetD.`

The residual derivation layer below derives the Appendix C targets one at a time from the package and then packages those derivations as:

`caie_appendix_C_residual_target_from_axiom_package.`

The first residual target has now been discharged. Isabelle proves:

`caie_appendix_C_first_residual_target_from_axiom_package.`

This is the local derivability statement

$\Gamma; \text{caie_appendix_C_axiom_package} \vdash_{\text{Caie}} \text{caie_Thm32}.$

The proof does not assume `caie_Thm32` itself. It assumes the component-box down-projection principle `caie_name_expanded_down_phae_components_principle`. The generic CEV helper lemmas

`CEV_imp_trans, CEV_imp_lift_right, CEV_curry_conj,`
`CEV_box_conj_intro_imp, CEV_box_conj3_from_components_imp`

assemble the three boxed components into the packed phae claim. The Caie-specific bridge is

`CEV_caie_name_expanded_down_phae_from_components,`

which derives the old expanded principle from the component principle. The formal proof also uses two generic de Bruijn helper facts, `weakening_after_front` and `subst0_Var0_rename_lift_Suc`, to instantiate the shifted universal component principle cleanly.

Finally, the file proves the conversion bridge

$\text{caie_Thm32} \simeq_{\beta\eta} \text{caie_name_expanded_down_phae_principle},$

and then uses the corresponding CEV implication to transfer local derivability to the official Appendix C statement.

The second residual target has also been discharged. Isabelle proves:

`caie_appendix_C_second_residual_target_from_axiom_package.`

This is the local derivability statement

$\Gamma; \text{caie_appendix_C_axiom_package} \vdash_{\text{Caie}} \text{caie_Thm33}.$

Again, the proof does not assume the Appendix C target itself. The package already contains five component principles saying that, if P is phae, then $\text{MC}(P^{\dagger c})$, $\text{AC}(P^{\dagger c})$, $\text{HP}(P^{\dagger c})$, $\text{NHP}(P^{\dagger c})$, and $\text{ACF}(P^{\dagger c})$ each hold under $\Box_o$. The file bundles these as

`caie_Thm33_component_principles.`

The new definitional bridge expands $\text{Name}(Q)$ into its boxed five-part conjunction:

`beta_eta_caie_Name.`

The bridge theorem

`CEV_caie_Name_from_component_boxes`

then says that the five boxed components imply $\text{Name}(Q)$. The generic five-component helpers

`CEV_shifted_forall_inst_Var0, CEV_imp_common_conj5_intro,`
`CEV_box_conj5_from_components_imp`

combine the five instantiated package principles and assemble the boxed Name-definition. The main global bridge is

`CEV_caie_Thm33_from_component_principles,`

and the package-level derivation is

`caie_CEV_derivable_Thm33_from_axiom_package.`

The next two residual equality targets are now derived from pointwise package princi-

ples. For Theorem 34 the package member is

`caie_phae_up_down_pointwise_principle.`

In Bacon/Caie notation it says that if P is a possible haecceity, then

$$\forall x : e. P(x) \leftrightarrow (P^{\dagger c})^{\downarrow c}(x).$$

It does not assert the official equality target

$$P =_{\text{Pr}} (P^{\dagger c})^{\downarrow c}.$$

Isabelle instantiates the pointwise principle with

`CEV_caie_phae_up_down_pointwise_instance,`

then uses contextual unary equivalence to close from fresh-argument agreement to property equality:

`CEV_caie_Thm34_from_up_down_pointwise,`
`caie_CEV_derivable_Thm34_from_axiom_package,`
`caie_appendix_C_fourth_residual_target_from_axiom_package.`

For Theorem 35 the package member is

`caie_name_down_up_pointwise_principle.`

In Bacon/Caie notation it says that if Q is a name, then

$$\forall P : \text{Pr}. Q(P) \leftrightarrow (Q^{\downarrow c})^{\dagger c}(P).$$

Again the package does not assume the official equality target

$$Q =_{\text{Cl}} (Q^{\downarrow c})^{\dagger c}.$$

The verified derivation is:

```
CEV_caie_name_down_up_pointwise_instance,
CEV_caie_Thm35_from_down_up_pointwise,
caie_CEV_derivable_Thm35_from_axiom_package,
caie_appendix_C_fifth_residual_target_from_axiom_package.
```

Both equality derivations use the contextual vector-equivalence rule in CEV, so they are no longer conditional on a separate admissibility assumption.

Theorem 36 is also derived from the package. The general zeta layer proves the theorem-level unary case of vector equivalence:

CEV_unary_equivalence.

In Bacon/Caie notation, this says that if $F, G : \sigma \rightarrow_o t$ agree on a fresh argument, then $F =_{\sigma \rightarrow_o t} G$. In Isabelle notation the pointwise premise has the form

$$\sigma \# \Gamma \vdash_{\text{CEV}} (\text{App } (\text{shift } F) (\text{Var } 0) \leftrightarrow \text{App } (\text{shift } G) (\text{Var } 0)),$$

and the conclusion is

$$\Gamma \vdash_{\text{CEV}} F =_{\sigma \rightarrow_o t} G.$$

The Caie file also derives small CEV infrastructure lemmas

CEV_forall_inst, CEV_modal_T_imp,

and proves the instantiated haecceity-up principle

CEV_caie_haecceity_up_extensionality_instance.

This last theorem instantiates the principle

caie_haecceity_up_extensionality_principle

at the current property P and individual x . In Bacon/Caie notation its conclusion is the implication

$$\text{phae}(P) \wedge P = \text{heq}(x) \rightarrow \Box_o \forall R : \text{Pr}. P^{\dagger c}(R) \leftrightarrow R(x).$$

This is exactly the pointwise material needed for Theorem 36 before applying higher-order extensionality.

The extra proof-theoretic ingredient is now part of CEV itself: CEV_proves has a con-

textual vector-equivalence rule. The Caie file derives the old named admissibility predicate as

`CEV_contextual_unary_equivalence_admissible_holds.`

In report notation, this says: if A is a formula and $F, G : \sigma \rightarrow_o t$, and under the shifted hypothesis A the fresh argument instances of F and G are biconditional, then A implies $F =_{\sigma \rightarrow_o t} G$. With that CEV rule, Isabelle proves

`CEV_caie_Thm36_contextual_unary_premise,`
`CEV_caie_Thm36_from_haecceity_up_extensionality,`
`caie_CEV_derivable_Thm36_from_axiom_package,`
`caie_appendix_C_third_residual_target_from_axiom_package.`

The proof does not assume `caie_Thm36`. It uses modal T , universal instantiation, beta-eta conversion for the classifier lambda term, the contextual vector-equivalence rule, and the package member

`caie_haecceity_up_extensionality_principle.`

Theorem 37 is now derived from the same package without the contextual unary-equivalence assumption. The added package member is the smaller witness principle

`caie_phae_up_hae_witness_principle.`

It is stated with the unfolded upward-projection body. In report notation:

$\forall P : \text{Pr}. \forall x : e. \text{phae}(P) \rightarrow (\text{caie_up_body } P \text{ heq}(x) \rightarrow \text{hae}(P)).$

Thus it is not the official Appendix C target, whose consequent is a negated application of $P^{\dagger c}$. The proof first specializes this principle:

`CEV_caie_phae_up_hae_witness_instance.`

It then uses the already verified beta-eta bridge for upward projection,

$P^{\dagger c}(R) \simeq_{\beta\eta} \text{caie_up_body } P \text{ } R,$

via `CEV_caie_up_apply`. With $R = \text{heq}(x)$, propositional contraposition yields

$$\text{phae}(P) \wedge \neg \text{hae}(P) \rightarrow \neg P^{\downarrow c}(\text{heq}(x)).$$

Finally, CEV generalization reintroduces the individual and property quantifiers. The verified theorems are:

```
CEV_caie_Thm37_from_up_hae_witness,
caie_CEV_derivable_Thm37_from_axiom_package,
caie_appendix_C_sixth_residual_target_from_axiom_package.
```

Theorem 38 is now also derived from the package. The package member is

```
caie_wname_dsim_surrogacy_principle.
```

It is a smaller surrogate for the official target: under weak-name hypotheses on Q and Z , its left side is the expanded equality

$$Q^{\downarrow c} =_{\text{Pr}} Z^{\downarrow c},$$

not the official down-similarity formula $Q \sim_{\downarrow c} Z$. The right side is Caie's boxed identity condition. The derivation then uses the verified definitional bridge

$$Q \sim_{\downarrow c} Z \simeq_{\beta\eta} Q^{\downarrow c} =_{\text{Pr}} Z^{\downarrow c}$$

via `CEV_caie_dsim`. The generic replacement helper used at this stage is `CEV_bicond_replace_left`. Isabelle proves the instantiated surrogate, the bridge to the official theorem, and the package-level result:

```
CEV_caie_wname_dsim_surrogacy_instance,
CEV_caie_Thm38_from_wname_dsim_surrogacy,
beta_eta_caie_Thm38_wname_dsim_surrogacy,
caie_CEV_derivable_Thm38_wname_dsim_surrogacy_bridge,
caie_CEV_derivable_Thm38_from_axiom_package,
caie_appendix_C_seventh_residual_target_from_axiom_package.
```

The final local derivation now consumes the package's surrogate axiom together with this whole-form bridge via `caie_CEV_derivable_bicond_right`.

Theorems 39 and 40 are now derived by the same method. Theorem 39 uses

`caie_hae_wname_dsim_identity_surrogacy_principle,`

which states the haecceity-like/weak-name identity biconditional with $Q^{\downarrow c} =_{Pr} Z^{\downarrow c}$ in place of $Q \sim_{\downarrow c} Z$. The proof uses `CEV_caie_dsim` and `CEV_bicond_replace_left` to recover the official statement, and also records `beta_eta_caie_Thm39_hae_wname_dsim_identity_surrogacy` and `caie_CEV_derivable_Thm39_hae_wname_dsim_identity_surrogacy_bridge` for the final derivability-level conversion. Theorem 40 uses

`caie_wname_unique_name_down_eq_surrogacy_principle.`

This package member has Caie’s unique-name existence shape, but both down-similarity atoms are replaced by equality of down-projections. The file adds general beta-eta congruence helpers for conjunctions, implications, object-language biconditionals, and quantifier bodies, then proves

`beta_eta_caie_Thm40_down_eq_surrogacy.`

From that beta-eta bridge, Isabelle derives the official theorem and the package-level result:

`CEV_caie_Thm40_of_down_eq_surrogacy,`
`caie_CEV_derivable_Thm40_down_eq_surrogacy_bridge,`
`caie_CEV_derivable_Thm40_from_axiom_package,`
`caie_appendix_C_ninth_residual_target_from_axiom_package.`

10.7 Appendix C statements

The file now contains exact object-language statements corresponding to Caie’s Appendix C theorems 32–40. The following table gives their report-level shape; the Isabelle definitions are the authoritative source for binder order and de Bruijn indexing.

- 32 $\forall Q : \text{Cl. Name}(Q) \rightarrow \text{phae}(Q^{\downarrow c})$
- 33 $\forall P : \text{Pr. phae}(P) \rightarrow \text{Name}(P^{\downarrow c})$
- 34 $\forall P : \text{Pr. phae}(P) \rightarrow P =_{\text{Pr}} (P^{\downarrow c})^{\downarrow c}$
- 35 $\forall Q : \text{Cl. Name}(Q) \rightarrow Q =_{\text{Cl}} (Q^{\downarrow c})^{\downarrow c}$
- 36 If P is a possible haecceity and $P = \text{heq}(x)$, then $P^{\downarrow c}$ is the classifier $\lambda R : \text{Pr. } R(x)$.
- 37 If P is a possible haecceity but not an actual haecceity, then $P^{\downarrow c}$ applies to no actual haecceity property $\text{heq}(x)$.
- 38 Weak names Q, Z are down-similar exactly under Caie's stated boxed identity condition.
- 39 A haecceity-like classifier and a weak name are down-similar iff they are identical.
- 40 Every weak name has a unique genuine name down-similar to it.

10.8 Current Appendix C dependency sort

The verified definitional material is the bridge package described above:

$$\begin{aligned}
 \text{heq}(x)(y) &\simeq_{\beta\eta} y =_e x, \\
 Q^{\downarrow c}(x) &\simeq_{\beta\eta} Q(\text{heq}(x)), \\
 P^{\downarrow c}(R) &\simeq_{\beta\eta} \text{caie_up_body } P \ R, \\
 Q \sim_{\downarrow c} Z &\simeq_{\beta\eta} Q^{\downarrow c} =_{\text{Pr}} Z^{\downarrow c}.
 \end{aligned}$$

The file packages these facts as:

`caie_CEV_derivable_definitional_bridge_package.`

This theorem says that all four bridges are available inside arbitrary $\Gamma; \Delta \vdash_{\text{Caie}}$ contexts, provided the displayed terms are well-typed. For the Appendix C axiom package itself, the specialization

`caie_appendix_C_axiom_package_definitional_bridge_package`

records the same four bridge facts over `caie_appendix_C_axiom_package`.

For the full Appendix C theorem statements, the current verified sort is as follows.

- Definition and beta-eta conversion alone: `caie_appendix_C_definitional_targets`, verified equal to the empty list.
- Explicit Caie-style principles required: `caie_appendix_C_principle_targets`, verified equal to `caie_appendix_C_theorems`.

Thus no full theorem among 32–40 is currently certified as a definitional-only consequence. All nine remain principle targets:

`{32, 33, 34, 35, 36, 37, 38, 39, 40}.`

This does not mean that all nine are still open. Theorems 32–40 are now verified from the explicit axiom package. They remain in the principle-target bucket only because those proofs use substantive Caie-style principles rather than definition and beta-eta conversion alone. The formal frontier is recorded by the two lists

```
caie_appendix_C_axiom_package_verified_targets =
    [32, 33, 34, 35, 36, 37, 38, 39, 40],
caie_appendix_C_axiom_package_remaining_targets = [].
```

The Isabelle lemmas `caie_appendix_C_sort_covers` and `caie_appendix_C_sort_disjoint` verify that this is a partition of the current definitional-vs-principle target list. The additional lemmas `caie_appendix_C_axiom_package_target_sort_covers` and `caie_appendix_C_axiom_package_target_sort_disjoint` verify the package-derived-vs-remaining split. Every residual target is proved well-typed, and each one is locally derivable from the explicit package.

This classification is a proof-status classification, not a semantic independence theorem. It says that the currently verified beta-eta bridge package does not by itself certify any complete Appendix C theorem statement. The explicit counterfactual/name/weak-name/haecceity package has now been introduced, and Theorems 32–40 have been derived from it using definitional bridge facts, propositional reasoning, and pointwise-to-equality closure, with Theorems 34–36 using the contextual vector-equivalence rule in CEV. The final conversion steps for Theorems 32 and 40 now use `caie_CEV_derivable_bicond_right` together with the named bridge facts, so those bridge applications remain inside the local Caie derivability layer. The theorem

```
caie_appendix_C_residual_target_from_axiom_package
```

is the collective residual-target layer: every member of `caie_appendix_C_theorems` is locally derivable from `caie_appendix_C_axiom_package`. The intermediate list

```
caie_appendix_C_residual_bridge_pairs
```

records, for each target, the source principle or source bundle from which the target is derived. The theorem

```
caie_appendix_C_residual_target_from_axiom_package_using_
    bridge_pairs
```

is the explicit source-to-target version of this result. Internally, it uses `caie_appendix_C_axiom_package_derives_residual_bridge_source` for the package derivation of the source side and `caie_CEV_derivable_residual_bridge_pair_imp` for the verified source-to-target implication, followed by local modus ponens in `caie_CEV_derivable`. The theorem

`caie_appendix_C_principle_target_from_axiom_package_using_bridges`

states the same conclusion over the verified principle-target bucket. The theorem `caie_appendix_C_axiom_package_closes_targets_from_residual_bridges`

then packages that bucket-level result as the closure predicate. The theorem `caie_appendix_C_axiom_package_closes_targets_from_verified` records that the package closes the entire Appendix C principle-target bucket.

10.9 The finite model M^* and Proposition 29

The session now includes a shallow HOL reconstruction of Caie's finite model M^* in `Bacon_Caie_Mstar.thy`. This file is separate from the deep-embedded proof system: it gives finite datatypes for the worlds, the two individuals, and the relevant property-domain elements, then defines the table interpretations `Astar` and `Pstar`.

The currently verified model facts are:

<code>Pstar_is_Name</code>	P^* satisfies the finite Name clauses,
<code>Astar_is_Name</code>	A^* satisfies the finite Name clauses,
<code>bullet3_P_is_empty</code>	P^* is empty at the actual world,
<code>Astar_nonempty</code>	A^* is nonempty at the relevant point,
<code>Mstar_name_coverage</code>	A^* and P^* cover the finite individuals,
<code>name_class_is_rigid</code>	the displayed Name class is rigid,
<code>name_class_boxed_coverage</code>	the displayed Name class covers each accessible individual,
<code>Mstar_name_plenitude_witness</code>	the finite rigid-class/plenitude witness,
<code>Mstar_proposition29_bullet1_from_NDS_soundness</code>	the conditional theorem-to-truth bridge for M^* ,
<code>Mstar_proposition29_core</code>	the earlier Name/emptiness/coverage package,
<code>Mstar_proposition29_finite_bullets</code>	the combined finite bullets 2-4 package,

`Mstar_proposition29_from_NDS_soundness` the conditional all-bullets package.

The new witness is `name_class`, a world-indexed class of classifiers whose members are exactly the two displayed M^* -Names up to their finite domain table behaviour. Isabelle verifies that this class is persistent and inextensible, hence rigid in the finite sense used here, and that at every world accessible from actuality each individual has one of its member Names applying to that individual's haecceity. Thus the finite M^* layer now proves Proposition 29's Name, emptiness, and plenitude bullets.

The file now also isolates Proposition 29's first bullet as an abstract soundness bridge. The definition `nds_background_soundness` represents the Proposition 27-style premise that every theorem of the background logic is true in every NDS model. The definition `mstar_nds_model_obligation` records the separate obligation that the finite structure M^* is an NDS model. From those two assumptions, the theorem

`Mstar_proposition29_bullet1_from_NDS_soundness`
proves theorem-truth at M^* . The theorem

`Mstar_proposition29_from_NDS_soundness`
combines that result with the finite bullets. This does not yet prove Proposition 27; it makes the remaining dependency exact.

11 What has intentionally not been done yet

Several important pieces of Bacon's framework are intentionally not yet formalized:

- no proof yet of `CEV_context_equiv_abstract_const_admissible`;
- no unconditional proof yet of the CE/CEV prefix internalization principles;
- no embedding of the term model into the earlier full-function `applicative_structure` locale; the current term model deliberately avoids that stronger lambda-denotation requirement;
- no full object-language/NDS-soundness proof yet proving `nds_background_soundness` for Caie's background logic;
- no general NDS-model soundness theorem yet for Caie's Proposition 27;

- no further decomposition yet of the current Caie surrogate package principles into still smaller philosophically natural components.

Those omissions are now quite narrow. The H/C Henkin construction and term-model completeness package are verified. The CE/CEV layer has been reduced to explicit closure and abstraction principles rather than left as a generic “build a canonical model” task.

12 Recommended next steps

There are now two natural tracks.

For the general Baconian completeness project, the next milestone is to prove

`CEV_context_equiv_abstract_const_admissible.`

Technically, this means proving enough CEV renaming/substitution invariance to push abstraction through theoremhood cases involving `zeta_body`. Once that is verified, the existing conditional countermodel-property theorems can be instantiated.

For the Caie application track, the next milestone is to connect the finite M^* witness layer to a general NDS-model soundness theorem for the background logic. The Isabelle file now names the two exact obligations: prove `nds_background_soundness` for Caie’s background theoremhood predicate and prove `mstar_nds_model_obligation` for the finite M^* structure. These two facts discharge Proposition 27 and Proposition 29’s first bullet through the verified bridge. After that, the remaining work is to refine the current Appendix C surrogate principles into still smaller philosophically natural components where appropriate.